

Final Exam Review - Spring 2026

Exam Details

- The exam is made up of:
 - Writing code to solve problems.
 - Tracing execution given specified input values.
- There are multiple versions of the exam.
- **Partial credit is available for the code-writing questions**
 - Your goal should be to not need it.
 - Focus your solution on correctness of the problem solving method. Clear evidence of your thought process and plan must be included in the code and comments in order to qualify for partial credit.
 - Put down anything you do know even if you can't solve the entire problem. Some partial credit is better than getting no credit.
 - Make your thinking visible so applying partial credit is easier.
- **Not allowed**
 - Any electronic devices (including calculators, phones, smart watches, and computers)
- **Allowed**
 - A writing utensil (e.g. pen/pencil)
 - Scratch Paper (8.5X11 inches)
 - Up to 5 pages of exam aids that you create.
 - Exam aids can be pages up to 8.5X11 inches and can be handwritten or printed on both sides.
 - You can't use exam aids for scratch work during the exam.
 - If you do, then you must submit the exam aid with your exam.
- We will be manually grading the exam, so don't get stressed if it is not perfect since you will get partial credit. We will also ignore minor errors that do not significantly impact the results.

Exam Strategies

- Understanding at a high level will help you code better during an exam. So, as you study, ask yourself...
 - *What is happening?*
 - *High level: What is the goal?*

- *Code level: What are the effects and consequences of statements, etc.?*
- *Why is it that way?*
 - *High level: Why are certain design decisions made? Why is it designed that way?*
 - *Code level: What is the code that accomplishes a higher-level goal? Why is it done that way?*
- *How is it done?*
 - *High level: How do you accomplish a high-level goal?*
 - *Code level: How do you write the code?*
- **Study Strategy: Prioritize how you study to maximize your ability to demonstrate concept mastery while not overwhelming yourself.**
 - For a solid passing grade focus on class topics, slides, examples, in-class activities, homework, and labwork.
 - Make sure you understand anything coded in homework, labwork, and in-class examples.
 - What general (higher level) principles were illustrated in a labwork/homework?
 - Could you explain it in general?
(i.e. what/why rather than how)
 - Be able to solve and write code for similar problems.
 - After you've done all you can do in the "for a solid passing grade" above, then review the readings.
 - After you've done all you can from the previous approaches and if you are still fuzzy about a topic, read about it in a different source or textbook. You can also talk to someone who can explain it in a different way.

Note, topics are not always organized sequentially

Exam Topics

The final exam is cumulative. This review sheet covers topics after Exam 3. You should look at prior review sheets to remind you of concepts from previous exams which may show up.

Linked List

- Singly and Doubly linked lists
- Structure
 - Head
 - [Tail]
 - Node Structure
- Know how to do basic operations for both types
 - Traversal
 - Insert / Remove
 - Front
 - Back
 - Middle

Inheritance and Polymorphism (Code Tracing questions only)

- Pros / Cons
- Class hierarchies
- Base class / superclass / parent class
- Derived class / subclass / child class
- Access
 - public / private / protected

- Data members /
Member functions /
Constructors
- const Member functions
- Overriding methods
 - Virtual Functions
- Abstract Classes (pure virtual)
 - Pure Virtual Functions
- When to use Virtual and Pure Virtual functions.
- Types of polymorphism
 - Ad-Hoc - function overloading
 - Subtyping - using derived class in place of base class
 - Parametric - templates
- How do you create a vector (or list, etc.) that holds elements of an abstract class?
- How do you create a vector (or list, etc.) that holds elements that all share the same base class?
- Function templates
- Class templates

Recursion (Code Tracing questions only)

- Base cases
- Recursive cases

Practice Problems

Note that practice problems are generally greater than or equal in difficulty to what you will find on the exam. Exam questions are designed to be challenging and doable in the time provided. Some of the practice questions are longer than what you would see on an exam, but should give you good practice.

Don't forget to review practice problems from review sheets for prior exams.

For linked list questions assume the following Linked list class definition:

```
struct Node {
    int value;
    Node* next;
    Node(int num = 0) : value{num}, next{nullptr} {}
};

class LinkedList {
    Node* head;

public:
    LinkedList() : head{nullptr} {}
    // other member functions
};
```

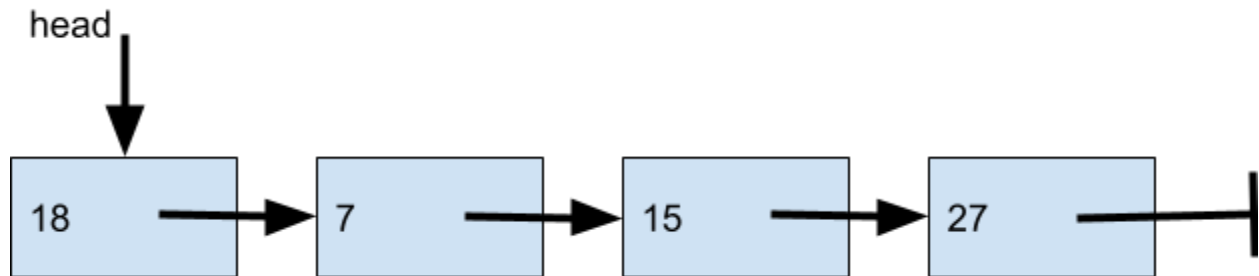
Linked List Average

Write a member function of the class `LinkedList` (a simply-linked list, with a pointer `head` to its first element) that returns the average of the elements in the list.

- Do not assume the existence of any other functions to use.
- Throw an exception if the list is empty.

Example

For the list



the function should return 16.75.

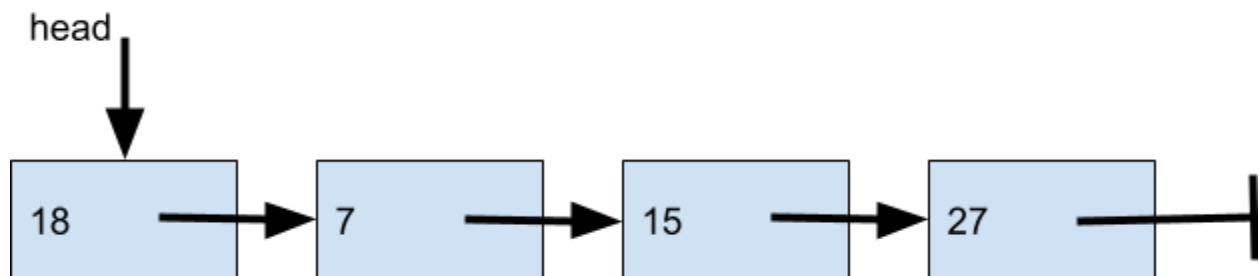
Remove all Maximum

Write a member function of the class LinkedList that removes from the list all occurrences of the maximum element in the list.

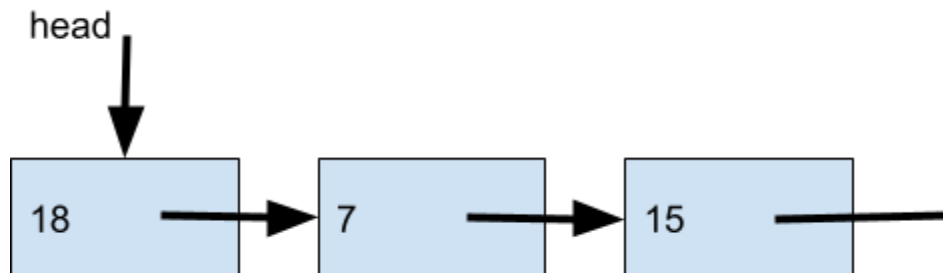
- Observe that the list can be and can become empty.
- Do not assume the existence of any other functions to use.

Example 1

The list

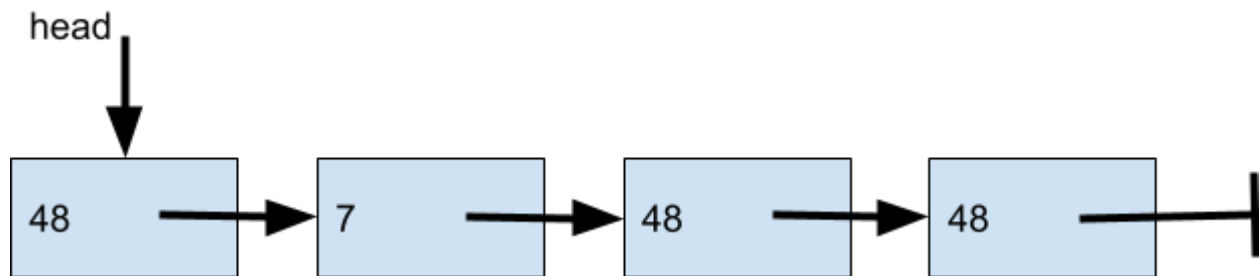


Will become

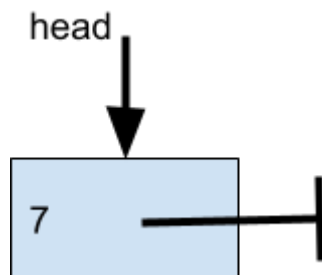


Example 2

The list



Will become



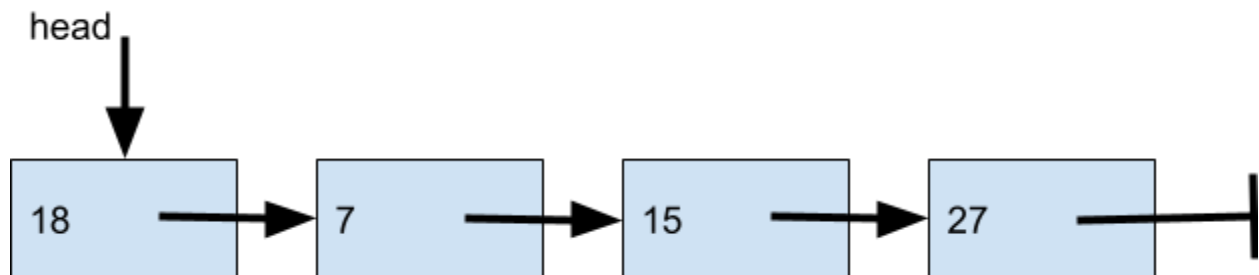
Reverse Linked List

Write a member function of the class LinkedList that reverses the list.

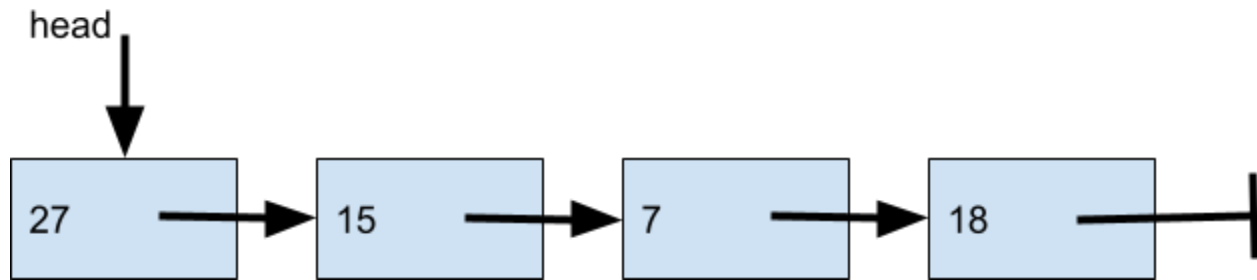
- Observe that the list can be empty.
- Do not assume the existence of any other functions to use.

Example

The list



Will become



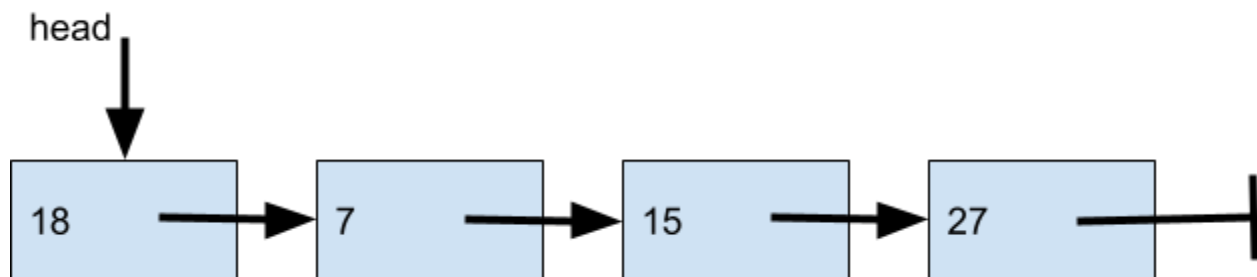
Middle of the List

Write a member function of the class `LinkedList` that returns a pointer to the element in the middle of the list (if the list has n elements, it should return a pointer to the $(\lfloor \frac{n}{2} \rfloor + 1)$ - *th* element).

- Do not assume the existence of any other functions to use.
- Return `nullptr` if the list is empty.

Example

For the list



The function should return a pointer to the node that contains 15.

Linked List Cycle

Write a member function of the class `LinkedList` that returns `true` if a cycle exists in the linked list, `false` otherwise.

- Do not assume the existence of any other functions to use.
- Return `false` if the list is empty.

What does “cycle” mean? The singly linked list is a linear sequence. That is, starting with

the *head*, if we traverse each node, no node will be visited twice. However, if the *next* pointer stored within a node points to an already traversed node, it will form a cycle (see example below).

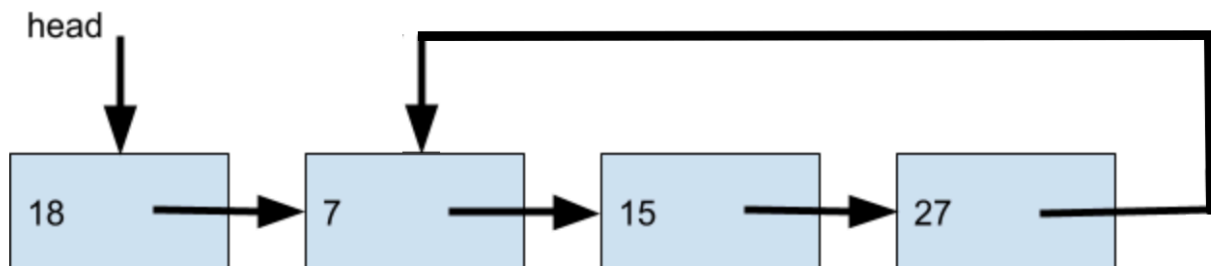
Hint

You need to maintain information about the traversed/visited nodes. A couple of options:

- [bad] Use a bool variable in the Node class (violates [open-closed principle](#))
- Use a collection (e.g., dynamic array or vector) to store the information about visited nodes (What information? What if values are not unique?)
- Use [Floyd's tortoise and hare](#): traverse the list with two pointers, the tortoise that goes in steps of 1 node, and the hare that goes in steps of 2 nodes.

Example

For the list



The function should return true.

Code Tracing

For each piece of code, assume that the file has the necessary `#include` and `using` statements to compile

Q1 What does this code print?

```
int foo(int n) {
    if(n == 0) {
        return 5;
    }
    return foo(n/3) + 1;
}
```



```
int main() {
    cout << foo(10);
}
```

Q2 What does this code print?

```
class Fruit {
private:
    string name;
protected:
    int weight;
public:
    Fruit() : name{" "}, weight{5} {}
    virtual string desc() {
        return std::to_string(weight);
    }
};

class Apple : public Fruit {
private:
    string color;
public:
    Apple(string s): color{s} {}
    string desc() override {
        return color;
    }
};

int main() {
    Apple a("Red");
    Fruit& fr = a;
    Fruit* fp = &a;
    cout << fr.desc() << ", " << fp->desc();
}
```

Q3 Answer the following questions about the code from Q2:

- Can Apple access Fruit::name?
- Can main access Fruit::name?
- Can Apple access Fruit::weight?
- Can main access Fruit::weight?

Q4 What does this code print?

```
class A {
    int x;
public:
    virtual void f() {
        cout << "A";
    }
}
```

```

};
class B : public A {
public:
    void f() {
        cout<< "B" ;
    }
};
int main() {
    B* ptr = new B();
    A x = *ptr;
    A& y = *ptr;
    A* z = ptr;
    x.f();
    y.f();
    z->f();
}

```

Q5 What does this code print?

```

class A {
public:
    void print() {
        cout << "A";
    }
};
class B : public A {
public:
    void print() {
        cout << "B";
    }
};
int main() {
    B x;
    A y = x;
    A& z = x;
    x.print();
    y.print();
    z.print();
}

```

Q6 What does this code print?

```

class A {
public:

```

```

    int x;
    A() : x{2} {}
    int f() {
        return x;
    }
};
class B : protected A {
    int y;
public:
    B() : y{5} {}
    virtual int f(){
        return y;
    }
};
class C : public B {
public:
    int z;
    C() : z{9} {}
    int f() {
        return z;
    }
};
int main() {
    C obj;
    cout << obj.f();
    B* ptr = &obj;
    cout << ptr->f();
}

```

Q7 Answer the following questions about the code from Q6:

- Can main access ptr->x?
- Can main access ptr->y?
- Can C access A::x?
- Can C access B::y?

Q8

```
class Animal {
    std::string name;

protected:
    int age;

public:
    Animal(std::string name, int age) : name(name), age(age) {}
    std::string getName() const { return name; }
};

int main(){
    Animal* a1 = new Animal("Tod", 10);
    Animal* a2 = new Animal("Tod", 10);
    Animal* a3 = a1;
    Animal a4 = *a2;
    //based on this point of the code
}
```

Based on the Animal class, select ****all**** of the following that are true.

- ☐ `a1 == a2` is true because the constructor for Animal initializes the attributes with Tod (name), and 10 (age) for the two objects (a1, a2).
- ☐ This code has memory leaks.
- ☐ name **can** be directly modified by a child/derived class.
- ☐ age **can** be directly modified by a child/derived class.
- ☐ `getName()` **can** be overridden by a child/derived class.
- ☐ If you include after the comment `"std::cout << a3.getName();"` the output will be "Tod"
- ☐ It makes sense in most cases for `a4` to be directly compared with `a3`, e.g. `a4 == a3`.
- ☐ It makes sense in most cases for `*a1` be directly compared with `*a3`, e.g. `*a1 == *a3`.
- ☐ Outputting variables of type Animal to an output stream using the output operator '`<<`' works as currently implemented, e.g. `"std::cout << a4"`.
- ☐ Constructing an instance of the Animal class, i.e. `"Animal a5;"`, works as currently implemented.
- ☐ Outputting the value returned by `getName` of the Animal class, i.e. `"std::cout << a4.getName();"`, works as currently implemented.
- ☐ Outputting the value returned by `getName` of the Animal class, i.e. `"std::cout << a2->getName();"`, as currently implemented.
- ☐ Outputting the age value of the Animal class, i.e. `"std::cout << a4.age;"`, works as currently implemented.
- ☐ Outputting the age value of the Animal class, i.e. `"std::cout << a2->age;"`, works as currently implemented.

Q9

```
class Base{
    char ch;
    public:
        Base() : ch('-') {
            std::cout << "Base class default constructor " << ch <<
std::endl;
        }
        Base(char c) : ch(c) {
            std::cout<<"Base class constructor "<< ch << std::endl;
        }
        virtual char getCh() const {return ch;};
        virtual void print() const = 0;
};

class Derived : public Base{
    char cc;
    public:
    Derived(){
        std::cout << "Derived class default constructor" <<
std::endl;
    }
    Derived(char c): Base(c) {
        std::cout <<"Derived class constructor " << std::endl;
    }
    void print()const{std::cout << "Print Derived Class" <<
std::endl;}
};

int main(){
    Derived d1;
    Derived d2('B');
    //based on this point of the code
}
```

Based on the Base and Derived classes, select ****all**** of the following that are true.

- ☐ Base is an abstract class
- ☐ Constructing an instance class Base as follows after the comment works as currently implemented. E.g. "Base b1('A');"
- ☐ Constructing an instance class Derived as follows after the comment works as currently implemented. E.g. "Derived d3('A');"
- ☐ The Derived class **MUST** implement the print() function.
- ☐ The Derived class **CAN** implement the print() function.
- ☐ The Derived class **MUST** implement the getCh() function.
- ☐ The Derived class **CAN** implement the getCh() function.
- ☐ The output for `Derived d1;` is

- Base class default constructor -
Derived class default constructor
- ☐ The output for `Derived d1;` is
Base class default constructor -
Derived class constructor
- ☐ The output for `Derived d1;` is
Base class constructor -
Derived class default constructor
- ☐ The output for `Derived d1;` is
Base class constructor -
Derived class constructor

Q10 What does this code print?

```
class Base {
public:
    virtual void show() { cout << "Base\n"; }
};

class Derived : public Base {
public:
    void show() override { cout << "Derived\n"; }
};

int main() {
    Base b;
    Derived d;

    Base *ptr1 = &b;
    Base *ptr2 = &d;

    ptr1->show();
    ptr2->show();
}
```

Q11 What does this code print?

```
class Person{
public:
    void virtual Name(){
        cout << "Human";
    }
}
```

```
};
```

```
class Bob : public Person{
public:
    void Name() override{
        cout << "Bob";
    }
};
```

```
class Joe : public Person{
public:
    void Name() override{
        cout << "Joe";
    }
};
```

```
int main()
{
    vector<Person*> people;

    people.push_back(new Person());
    people.push_back(new Bob());
    people.push_back(new Joe());

    for(int i = 0; i < people.size(); i++){
        people.at(i)->Name();
        cout << " ";
    }
}
```

Q12 What does this code print? Does it have a memory leak?

```
class smart_pointer{
    int* data;
    int* ref_count;
public:
    smart_pointer(int x)
        : data{new int{x}}
        , ref_count{new int{1}} {}
    smart_pointer(smart_pointer const& source)
        : data{source.data}
```

```

        , ref_count{source.ref_count} {
            (*ref_count)++;
        }
~smart_pointer() {
    (*ref_count)--;
    If (ref_count == 0) {
        delete data;
        delete ref_count;
    }
}
int& get_int() {
    return *data;
}
};

int main() {
    smart_pointer p(2);
    smart_pointer q = p;
    p.get_int() = 8;
    cout << q.get_int();
}

```

Q13 What does this code print?

```

template <typename T>
double func(T x){
    return x / 2;
}

int main(){
    cout << func(3) << " ";
    cout << func(3.0) << " ";
    cout << func<double>(3.0);
}

```


Q14 What does this code print?

```
template <typename T, typename U>
class dict {
    vector<T> keys = {};
    vector<U> values = {};

public:
    U& at(T val) {
        for (int i = 0; i < keys.size(); i++) {
            if (keys.at(i) == val) {
                return values.at(i);
            }
        }
        keys.push_back(val);
        values.push_back({});
        return values.back();
    }
};

int main() {
    dict<string, int> word_counts;
    word_counts.at("Hi") = 3;
    word_counts.at("Hello") = 5;
    word_counts.at("Hi") += 3;
    cout << word_counts.at("Hi") + word_counts.at("Hello");
}
```

Q15 What does this code print? Does it have a memory leak?

```
class base{
    int* x;
public:
    base(): x{new int{7}} {}
    base(int i): x{new int{i}} {}
    virtual double get_val(){
        return *x;
    }
    ~base() {
        delete x;
    }
};

class derived : public base{
    double* y;
public:
```

```
    derived(double j): y{new double{j}} {}  
    double get_val() override{  
        return *y;  
    }  
    ~derived(){  
        delete y;  
    }  
};  
int main(){  
    derived* d = new derived{2.0};  
    cout << d->get_val();  
    base* bs = d;  
    cout << bs->get_val();  
    delete bs;  
}
```

Q16 What are the base case(s) and recursive step(s)? What does the code output?

```
int foo(int n) {  
    if (n % 2 == 0) {  
        return 1 + foo(n/2);  
    }  
    if (n == 1) {  
        return 1;  
    }  
    return 1 + foo(3*n + 1);  
}  
  
int main() {  
    cout << foo(3);  
}
```